SWE5001 — MOCK EXAMINATION 1（中英对照版）

Architecting Scalable Systems · Case: GreenCharge EV Charging Platform
案例：GreenCharge 电动车充电平台

Time allowed: 3 hours (plus 30 minutes reading time) · Total: 50 marks · Open book (printed materials only)
考试时间 3 小时（另加 30 分钟阅读时间）· 总分 50 · 开卷（只能带纸质资料）
Answer ALL questions. Refer to the case study in Appendix A. Where a question asks for a diagram, draw it. State any assumption you make.
回答全部题目。参考附录 A 的案例。题目要求画图的必须画图。你做的任何假设都要写出来。
Practice paper written for self-study. It is not an official paper. 自学用练习卷，不是官方试卷。正式考试是纯英文，先做英文版，看不懂再对照中文。

Section A (25 marks) · A 部分（25 分）

Question 1 · 第 1 题

Refer to the case study in Appendix A.

> 参考附录 A 的案例。

(a)

(i) For use case UC3 (Start / Stop Charging Session), the test team plans to run a 10-minute test against the staging environment and to pass the API if the **average** response time is below 2 seconds. Do you agree with this pass criterion? Give **one** reason and propose **one** more reliable way to test this requirement. **(2 marks)**

> (i) 对于用例 UC3（开始 / 结束充电会话），测试团队计划在预发布环境跑 10 分钟测试，只要**平均**响应时间低于 2 秒就判 API 通过。你同意这个通过标准吗？给出**一个**理由，并提出**一个**更可靠的测试方法。 **(2 分)**

(ii) The operator dashboard (UC6, refreshed every 5 minutes) and the monthly settlement (UC7) both use the meter values sent by the charger units. A team member proposes to deliver each MeterReading to the component that computes the operator dashboard through a **message queue**. As the lead architect, you argue for **event streaming** instead. Give **two** reasons. **(2 marks)**

> (ii) 运营商看板（UC6，每 5 分钟刷新）和月度结算（UC7）都要用充电桩发来的电表读数。一位组员提议用**消息队列**把每条 MeterReading 送给计算看板的组件。作为首席架构师，你主张改用**事件流**。给出**两个**理由。 **(2 分)**

(b) Refer to the domain model in Figure 5 of Appendix A. Answer each part independently, based on the **original** Figure 5. For each concern below, **identify** the most likely cause, **propose** a countermeasure by changing the domain model, and state the **main drawback** of your countermeasure (if any).

> **(b)** 参考附录 A 图 5 的领域模型。每一小问都**以原图为准、独立作答**。对下面每个问题：**指出**最可能的原因，**提出**通过修改领域模型的对策，并说明对策的**主要缺点**（如果有）。

(i) The operations team reports that every StationManager instance uses a very large amount of runtime memory, and the memory keeps growing during the day. **(5 marks)**

> (i) 运维团队报告：每个 StationManager 实例都占用大量运行内存，而且一天里内存一直在涨。 **(5 分)**

(ii) Drivers complain that the charger availability shown in the Find Charger map (UC1) is often out of date; Section 4 requires availability changes to be visible within 10 seconds. **(4 marks)**

> (ii) 司机抱怨"找充电桩"地图（UC1）上显示的可用状态经常过时；第 4 节要求可用状态的变化在 10 秒内可见。 **(4 分)**

(c) Each charger unit currently calls the following API whenever its status changes and every 30 seconds while charging:

> **(c)** 目前每台充电桩在状态变化时、以及充电期间每 30 秒，都会调用下面这个 API：

github.com/bibi45c · 免费分享，禁止售卖

URI	HTTP	Input 输入	Output 输出
/chargers/status	PUT	stationId, connectorId, status, meterValue, tariff, driverId	"updated" / "not updated" "已更新" / "未更新"

Before replying, the platform recomputes the availability map of the whole city. In a load test with 15,000 simulated charger units, status updates are delayed by 15 to 40 seconds. Explain the likely **causes** of the delay, describe how you would **refactor** the API, and illustrate the new design with a **concrete example**. **(6 marks)**

> 平台在回复之前，要重新计算全市的可用状态地图。在模拟 15,000 台充电桩的负载测试中，状态更新延迟了 15 到 40 秒。解释延迟的可能**原因**，说明你会怎样**重构**这个 API，并用一个**具体例子**展示新设计。 **(6 分)**

(d) Identify the **two** most important quality attributes for the data store that keeps charger status and meter values in Phase 1, and explain why they are more important than other quality attributes. **(2 marks)**

> **(d)** 指出第一阶段中，保存充电桩状态和电表读数的数据存储**最重要的两个**质量属性，并解释为什么它们比其他质量属性更重要。 **(2 分)**

(e) Based on your answer to (d), propose **two** cloud database or storage services from one cloud provider, and explain how each helps the architecture meet those quality attributes. **(4 marks)**

> **(e)** 根据你在 (d) 的答案，从同一家云厂商提出**两个**云数据库或存储服务，并解释每个服务如何帮助架构满足这些质量属性。 **(4 分)**

Section B (13 marks) · B 部分（13 分）

Question 2 · 第 2 题

(a) The team has built the backend of Phase 1 as a single monolithic application. Propose a **hybrid cloud-native reference architecture** for the GreenCharge backend (you may consider microservices, scheduled jobs, event- or stream-driven services and serverless functions).

> **(a)** 团队把第一阶段的后端做成了一个单体应用。请为 GreenCharge 后端提出一个**混合云原生参考架构**（可以考虑微服务、定时任务、事件或流驱动的服务、无服务器函数）。

(i) Draw the main components and their interactions from a cloud perspective. **(3 marks)**

> (i) 从云的视角画出主要组件及它们之间的交互。 **(3 分)**

(ii) Apart from external calls through the API gateway, explain how the internal services communicate, and mark these communication components on your diagram. **(2 marks)**

> (ii) 除了经 API 网关的外部调用，解释内部服务之间怎样通信，并在图上标出这些通信组件。 **(2 分)**

(iii) Identify **one** service that needs to keep state and propose a suitable state-management mechanism. **(1 mark)**

> (iii) 指出**一个**需要保存状态的服务，并提出合适的状态管理机制。 **(1 分)**

(iv) Describe the overall deployment plan on Kubernetes, either as a diagram or as a short description. **(2 marks)**

> (iv) 用图或简短文字描述在 Kubernetes 上的整体部署计划。 **(2 分)**

(b) In Phase 2, UC9 (Ask Trip Assistant) sends the driver's question to an **external LLM provider API**. In tests the LLM API takes 3 to 8 seconds to answer and about 2% of the calls fail. Section 4 requires the app to acknowledge each question within 2 seconds for 90% of the requests and to show the answer within 15 seconds. Propose a design for UC9 that meets these requirements, and justify **two** of your design decisions. **(3 marks)**

> **(b)** 第二阶段，UC9（询问行程助手）会把司机的问题发给**外部 LLM 服务商的 API**。测试中 LLM API 需要 3 到 8 秒才回答，约 2% 的调用会失败。第 4 节要求：90% 的请求要在 2 秒内确认"已收到问题"，并在 15 秒内显示答案。为 UC9 提出满足这些要求的设计，并论证其中**两个**设计决策。 **(3 分)**

(c) The services will be deployed in containers. Because of a licence constraint of the container platform, two or more services must share one container instance. Services with the highest **communication affinity** (the number and size of messages exchanged over time) should be collocated. Based on the original Figure 5 and the use cases, propose the most suitable services to be collocated and explain the benefit. **(2 marks)**

> **(c)** 服务将以容器部署。由于容器平台的许可证限制，两个或更多服务必须共用一个容器实例。**通信亲和度**最高（一段时间内交换的消息数量和大小最大）的服务应该放在一起。根据原始图 5 和用例，提出最适合放在一起的服务，并说明好处。 **(2 分)**

github.com/bibi45c · 免费分享，禁止售卖

Section C (12 marks) · C 部分（12 分）

Question 3 · 第 3 题

(a) Refer to the domain model in Figure 5 and the logical packaging and deployment model in Figure 6 of Appendix A. **Identify two** undesirable architectural decisions in Figure 6, propose the corrective action for each, and explain how the decision leads to the corrective action. You may draw only the part of the diagram that is needed. **(6 marks)**

> **(a)** 参考附录 A 图 5 的领域模型和图 6 的逻辑打包与部署模型。**指出**图 6 中**两个**不理想的架构决策，为每个提出纠正措施，并解释这个决策为什么导致这样纠正。可以只画需要的那部分图。 **(6 分)**

(b) The CPO administrators propose to use **"the number of charging sessions per station per day"** as the application metric to evaluate both the **availability** of the charging stations and the **experience** of the drivers. Do you agree? Justify your answer. If you disagree, propose **two** better metrics, one for each concern, and justify each of them. **(4 marks)**

> **(b)** 运营商管理员提议用**"每个充电站每天的充电会话数"**作为应用指标，同时评估充电站的**可用性**和司机的**体验**。你同意吗？论证你的答案。如果不同意，提出**两个**更好的指标（可用性、体验各一个），并分别论证。 **(4 分)**

(c) The **Stop Charging Session** request passes through the following components **in series**: API gateway (99.95%), SessionManager service (99.9%), PaymentManager service (99.9%) and the external Payment Gateway (99.5%). Calculate the end-to-end availability, state whether it meets a target of 99.5%, and propose **one** change to the architecture that would improve it. **(2 marks)**

> **(c)** **"结束充电会话"**请求**串联**经过以下组件：API 网关（99.95%）、SessionManager 服务（99.9%）、PaymentManager 服务（99.9%）、外部支付网关（99.5%）。计算端到端可用性，说明是否达到 99.5% 的目标，并提出**一个**能提高可用性的架构改动。 **(2 分)**

— END OF PAPER · 试卷结束 —

github.com/bibi45c · 免费分享，禁止售卖

Appendix A — Case Study: GreenCharge EV Charging Platform · 附录 A 案例：GreenCharge 电动车充电平台

1. Background · 背景

The city's energy agency and several private **charge point operators (CPOs)** are building **GreenCharge**, a platform on which electric-vehicle (EV) drivers find, reserve, start and pay for public charging points operated by many different CPOs. Phase 1 delivers the platform with two pilot applications: the **Driver mobile app** and the **Operator web portal**. Phase 2 adds smart charging and an AI trip assistant.

> 城市能源局和几家私营**充电运营商（CPO）**正在建设 **GreenCharge** 平台。电动车司机在上面查找、预约、启动公共充电桩并付费，这些充电桩由许多不同的运营商经营。第一阶段交付平台和两个试点应用：**司机手机 App** 和**运营商网页门户**。第二阶段增加智能充电和 AI 行程助手。

Roles: **Driver**, **CPO Admin**, **Platform Admin**. External systems: **Payment Gateway**, **Grid Operator Price API** (publishes the electricity price every 15 minutes), **LLM Provider API** (Phase 2). About 15,000 **charger units** are installed in the city; each unit reports its status and meter values to the platform.

> 角色：**司机**、**运营商管理员**、**平台管理员。**外部系统：**支付网关**、**电网电价 API**（每 15 分钟发布一次电价）、**LLM 服务商 API**（第二阶段）。全市约安装了 15,000 台**充电桩**，每台都向平台上报状态和电表读数。

2. Phases and Use Cases · 阶段与用例

Phase 1 · 第一阶段

ID	Use case 用例	Description 说明
UC1	Find Charger 找充电桩	The driver views a map of nearby chargers with their availability (Available / Occupied / Faulted) and price. 司机查看附近充电桩的地图，显示可用状态（可用 / 占用 / 故障）和价格。
UC2	Reserve Charger 预约充电桩	The driver reserves a connector for up to 15 minutes. The platform sends a reservation notification. 司机预约一个充电枪口，最长保留 15 分钟。平台发送预约通知。
UC3	Start / Stop Charging Session 开始 / 结束充电会话	The driver scans the QR code on the charger. The platform authorises the driver and starts the session on the charger unit. While charging, the unit sends a meter value every 30 seconds. When the driver stops, the platform computes the cost, captures the payment through the Payment Gateway and issues a receipt. **The receipt is kept on the driver's phone so that it can be viewed offline.** 司机扫描充电桩上的二维码。平台给司机授权，并在充电桩上启动会话。充电期间，充电桩每 30 秒发一次电表读数。司机结束时，平台计算费用，经支付网关扣款并开收据。**收据保存在司机手机上，离线也能查看。**
UC4	Report Charger Status 上报充电桩状态	Each charger unit reports every status change and, while charging, a meter value every 30 seconds. 每台充电桩上报每一次状态变化；充电期间每 30 秒上报一次电表读数。
UC5	Manage Stations 管理充电站	The CPO Admin configures stations, connectors and tariffs. 运营商管理员配置充电站、充电枪口和电价。
UC6	View Operator Dashboard 查看运营商看板	The CPO Admin views utilisation and revenue per station, updated every 5 minutes. 运营商管理员查看每个站的使用率和收入，每 5 分钟更新。
UC7	Monthly Settlement 月度结算	At the end of each month the platform computes the revenue share of each CPO from all sessions and meter values of the month (batch). 每月底平台根据当月所有会话和电表读数，计算每个运营商的收入分成（批处理）。

Phase 2 · 第二阶段

github.com/bibi45c · 免费分享，禁止售卖

ID	Use case 用例	Description 说明
UC8	Smart Charging 智能充电	Every 15 minutes the platform adjusts the charging power of active sessions using the price from the Grid Operator Price API. 平台每 15 分钟按电网电价 API 的电价，调整进行中会话的充电功率。
UC9	Ask Trip Assistant 询问行程助手	The driver asks a question in natural language (e.g. "plan my charging stops to Malacca"); the platform sends it with the driver's context to the LLM Provider API and shows the answer. 司机用自然语言提问（如"规划去马六甲沿途的充电站"）；平台把问题连同司机的上下文发给 LLM 服务商 API，并显示答案。

3. Extensibility · 可扩展性

The platform shall provide common services and libraries so that future platforms of the agency (e-scooter sharing, battery swapping, public parking) can reuse them. In particular, notification and payment shall be reusable by the agency's other platforms.

> 平台应提供公共服务和库，让能源局未来的平台（电动滑板车共享、换电、公共停车）可以复用。特别是通知和支付，要能被能源局的其他平台复用。

4. Performance and Scalability · 性能与可扩展性

- Phase 1 shall support **200,000 registered drivers**, with up to **20,000 concurrent drivers** at the evening peak (6 pm to 9 pm), **300 CPO Admins** and **15,000 charger units**.
  第一阶段要支持 **20 万注册司机**，晚高峰（晚 6 点到 9 点）最多 **2 万并发司机**，**300 名运营商管理员**和 **15,000 台充电桩**。
- Interactive requests (find, reserve, start, stop) shall complete within **2 seconds for 90%** of the requests.
  交互请求（查找、预约、开始、结束）**90% 要在 2 秒内**完成。
- A change in charger availability shall be visible to drivers within **10 seconds**.
  充电桩可用状态的变化要在 **10 秒内**让司机看到。
- The operator dashboard shall be updated every **5 minutes**.
  运营商看板每 **5 分钟**更新一次。
- Phase 2: when a driver asks the Trip Assistant (UC9), the app shall show that the question has been received within **2 seconds for 90%** of the requests; the full answer shall be shown within **15 seconds** for 90% of the questions.
  第二阶段：司机询问行程助手（UC9）时，App **90% 要在 2 秒内**显示"问题已收到"；90% 的问题要在 **15 秒内**显示完整答案。
- Multitenancy is **not** expected in Phase 1; CPO data is separated by role-based access control.
  第一阶段**不要求**多租户；运营商数据靠基于角色的访问控制分开。

5. Security · 安全

The platform follows the DMZ / reverse-proxy practice. Personal data is encrypted at rest and in transit and handled according to the PDPA. Payment card data is handled only by the Payment Gateway and by as few platform subsystems as possible (PCI-DSS).

> 平台遵循 DMZ / 反向代理的做法。个人数据在存储和传输中都加密，并按新加坡个人数据保护法（PDPA）处理。支付卡数据只由支付网关和尽量少的平台子系统处理（PCI-DSS）。

6. Initial Architecture · 初始架构

The logical architecture uses the layers **Presentation**, **Control**, **Data Access**, **External Integration** and **Shared Library**. The Driver mobile app (with local storage) and the Operator web portal are the front ends; the backend runs in the cloud.

> 逻辑架构分为以下几层：**Presentation（表现层）**、**Control（控制层）**、**Data Access（数据访问层）**、**External Integration（外部集成层）**、**Shared Library（共享库层）**。司机手机 App（带本地存储）和运营商网页门户是前端；后端运行在云上。

7. Domain Model (Figure 5) · 领域模型（图 5）

Each box is a `<<service>>`. Entities in brackets [ ] with the same letter belong to one **aggregate** (red circle in the original diagram).

github.com/bibi45c · 免费分享，禁止售卖

每个框是一个 `<<service>>`（服务）。方括号 [ ] 里同一个字母的实体属于同一个**聚合**（原图中的红圈）。

<>	**Entities** 实体	**Aggregates** 聚合
DriverManager	Driver, Vehicle, PaymentMethodToken 司机、车辆、支付方式令牌	[D] Driver + Vehicle
StationManager	ChargingStation, Connector, Tariff, ChargerStatus, MeterReading 充电站、充电枪口、电价、充电桩状态、电表读数	**[S] ChargingStation + Connector + Tariff + ChargerStatus + MeterReading** (ChargingStation is the root) （ChargingStation 是聚合根）
SessionManager	ChargingSession, Reservation, Receipt 充电会话、预约、收据	[C] ChargingSession + Receipt; [R] Reservation
AvailabilityManager	AvailabilityMap, ConnectorAvailability 可用状态地图、枪口可用状态	[A] AvailabilityMap + ConnectorAvailability
PaymentManager	Payment, Refund 支付、退款	[P] Payment + Refund
BillingManager	Invoice, Settlement, RevenueShare 发票、结算、收入分成	[B] Settlement + RevenueShare; [I] Invoice
NotificationManager	NotificationMessage 通知消息	[N] NotificationMessage

Associations that cross service boxes · 跨服务框的关联：

- ChargingSession — Connector (StationManager): a session uses one connector.
  一个充电会话使用一个充电枪口。
- ChargingSession — Driver (DriverManager); ChargingSession — Payment (PaymentManager).
  充电会话关联司机（DriverManager）和支付（PaymentManager）。
- **ConnectorAvailability — ChargerStatus (StationManager): the availability of each connector is derived from the latest ChargerStatus.**
  每个枪口的可用状态由最新的 ChargerStatus 推算出来。
- Invoice — ChargingSession (SessionManager); Settlement — MeterReading (StationManager).
  发票关联充电会话（SessionManager）；结算关联电表读数（StationManager）。

8. Logical Packaging and Deployment Model (Figure 6) · 逻辑打包与部署模型（图 6）

Figure 6 shows the packaging and deployment for **UC3 Start / Stop Charging Session** only. It was drawn by a junior architect; it is **incomplete and may contain errors**. The deployment of the backend **Data Access subsystems and data stores** is **deliberately omitted**.

图 6 只画了 **UC3 开始 / 结束充电会话**的打包与部署。它由一位初级架构师绘制，**不完整，可能有错**。后端 **Data Access 子系统和数据存储**的部署是**刻意省略**的（没画不算错）。

Subsystem 子系统	**Manifests (packaged elements)** 打包进去的元素
Session Manager Presentation	StartSessionScreen «boundary», ReceiptScreen «boundary»
Session Manager Control	SessionController «control», PaymentGatewayAdapter «boundary»
Session Manager Data Access	ChargingSession «entity», Receipt «entity»
Payment Manager Control	PaymentController «control»
Payment Manager Data Access	Payment «entity», MeterReading «entity»
Notification Manager Control	NotificationController «control»

Node (location) 节点（位置）	**Deployed subsystems** 部署在上面的子系统
Mobile Device with Local Storage (Internet) 带本地存储的手机（互联网侧）	Session Manager Presentation, Payment Manager Control

github.com/bibi45c · 免费分享，禁止售卖

Node (location) 节点（位置）	**Deployed subsystems** 部署在上面的子系统
Application Server (Internal Zone) 应用服务器（内部区）	Session Manager Control, Notification Manager Control

github.com/bibi45c · 免费分享，禁止售卖

SWE5001 — MOCK EXAMINATION 2（中英对照版）

Architecting Scalable Systems · Case: TixNow National Ticketing Platform

案例：TixNow 全国票务平台

Time allowed: 3 hours (plus 30 minutes reading time) · Total: 50 marks · Open book (printed materials only)
考试时间 3 小时（另加 30 分钟阅读时间）· 总分 50 · 开卷（只能带纸质资料）
Answer ALL questions. Refer to the case study in Appendix A. Where a question asks for a diagram, draw it. State any assumption you make.
回答全部题目。参考附录 A 的案例。题目要求画图的必须画图。你做的任何假设都要写出来。
Practice paper written for self-study. It is not an official paper. 自学用练习卷，不是官方试卷。正式考试是纯英文，先做英文版，看不懂再对照中文。

Section A (25 marks) · A 部分（25 分）

Question 1 · 第 1 题

Refer to the case study in Appendix A.

> 参考附录 A 的案例。

(a) Two of the seeds of TixNow are **event tickets** and **seats**; the interactions include browsing events, joining the on-sale queue, selecting seats, paying and reselling tickets. With reference to the seeds and interactions, explain **one** architectural aspect that is most crucial for the success of the platform. **(2 marks)**

> **(a)** TixNow 的两个种子是**活动门票**和**座位**；交互包括浏览活动、加入开售排队、选座、付款和转售门票。结合种子和交互，解释对平台成功最关键的**一个**架构方面。 **(2 分)**

(b) Domain entities are an important part of the reusable assets of the platform.

> **(b)** 领域实体是平台可复用资产的重要组成部分。

(i) Analyse the use cases UC2, UC3, UC4 and UC6 in Section 5 and use the DDD approach to identify the domain objects and their relationships. Draw them as a UML class diagram, grouped into services. Do **not** base your answer on Figure 5; Figure 5 is used only in Q3(b). **(5 marks)**

> (i) 分析第 5 节的用例 UC2、UC3、UC4、UC6，用 DDD 方法找出领域对象及其关系，画成按服务分组的 UML 类图。**不要**以图 5 为依据；图 5 只用于第 3(b) 题。 **(5 分)**

(ii) An important feature of the platform is to have services that can be shared across the four applications and exposed to external applications. Identify the recommended **shared services** of the platform. **(4 marks)**

> (ii) 平台的一个重要特点是：有些服务能被四个应用共享，并开放给外部应用使用。找出平台推荐的**共享服务**。 **(4 分)**

(c)

(i) From the shared services in (b)(ii), select **two** to be exposed as APIs to **any two types** of external clients (producers or consumers). Name the clients and justify your choice. **(2 marks)**

> (i) 从 (b)(ii) 的共享服务里选**两个**，作为 API 开放给**任意两类**外部客户（生产者或消费者）。点名是哪些客户，并论证你的选择。 **(2 分)**

(ii) Select **one** of the services in (c)(i) and design its API at **Level 3** of the Richardson Maturity Model. Specify the service URIs, the HTTP actions, the input and output definitions, and explain the purpose of each operation. **(4 marks)**

> (ii) 从 (c)(i) 的服务里选**一个**，按 Richardson 成熟度模型 **Level 3** 设计它的 API。写出服务 URI、HTTP 动作、输入输出定义，并解释每个操作的用途。 **(4 分)**

(d) Propose a suitable type of data store and a cloud database or storage service for **each** of the following, and justify each choice in one or two sentences: **(4 marks)**

> **(d)** 为下面**每一项**提出合适的数据存储类型和一个云数据库或存储服务，每个选择用一两句话论证： **(4 分)**

github.com/bibi45c · 免费分享，禁止售卖

1. the event catalogue of UC1, where concerts, football matches and theatre shows have different attributes;
   UC1 的活动目录：演唱会、足球赛、话剧的字段各不相同；
2. the seat holds of UC2, which must expire automatically after 8 minutes;
   UC2 的座位保留：8 分钟后必须自动失效；
3. the orders and payments of UC3;
   UC3 的订单和支付；
4. the counters of the organiser sales dashboard of UC7, updated every minute.
   UC7 主办方销售看板的计数器，每分钟更新。

(e) Gate scanning (UC5) must keep working when the venue network is unstable.

> **(e)** 场馆网络不稳定时，闸口验票（UC5）也必须能继续工作。

(i) Explain how the scanner application and the backend should be designed so that 60,000 fans can enter within 90 minutes with each scan validated in less than 1 second even when the network is down. **(2 marks)**

> (i) 解释扫描器应用和后端应该怎样设计，才能让 6 万名观众在 90 分钟内入场，并且即使网络断了，每次扫描也能在 1 秒内完成校验。 **(2 分)**

(ii) State **one** risk of the design you proposed in (i), and explain how you would reduce it. **(2 marks)**

> (ii) 说出你在 (i) 提出的设计的**一个**风险，并解释你会怎样降低它。 **(2 分)**

Section B (13 marks) · B 部分（13 分）

Question 2 · 第 2 题

(a) A junior architect drew the following **logical detailed deployment diagram** for TixNow Concerts and TixNow Sports (functional components are not shown).

> **(a)** 一位初级架构师为 TixNow Concerts 和 TixNow Sports 画了下面这张**逻辑详细部署图**（没有画功能组件）。

Location 位置	**Node** 节点	**Subsystems deployed on the node** 部署在节点上的子系统
Internal Zone 1 内部区 1	Application node 应用节点	Concerts Presentation, Concerts Control, Sports Presentation, Seat Map Service 演唱会表现层、演唱会控制层、体育表现层、座位图服务
Internal Zone 1	Platform node 平台节点	Concert Recommendation Service, Virtual Queue Service, Ticket Issuing Service, Payment Service (calls the external Payment Gateway directly) 演唱会推荐服务、虚拟排队服务、出票服务、支付服务（直接调用外部支付网关）
Internal Zone 1	Data Store node 数据存储节点	Data Subsystem (one database shared by all services and all organisers) 数据子系统（所有服务和所有主办方共用一个数据库）
Internal Zone 1	Integration node 集成节点	Integration Subsystem 集成子系统

Notes: Seat Map Service is used by Concerts, Sports and the future theatre application. Concert Recommendation Service is used only by TixNow Concerts. There is no other location.

> 说明：座位图服务被 Concerts、Sports 以及将来的剧场应用使用。演唱会推荐服务只给 TixNow Concerts 用。图中没有其他位置（区域）。

Identify three undesirable architectural decisions in this diagram. For each, propose the corrective action and justify it. You may draw a similar diagram to illustrate. **(6 marks)**

> **指出**这张图中**三个**不理想的架构决策。对每一个，提出纠正措施并论证。可以画一张类似的图来说明。 **(6 分)**

(b) Based on your corrected diagram in (a), draw the **physical detailed deployment diagram** of TixNow on AWS using **some** of the following services: Amazon API Gateway, Amazon CloudFront, Amazon S3, AWS WAF, Amazon Cognito, Amazon EKS, AWS Lambda, Amazon SQS, Amazon DynamoDB, Amazon ElastiCache, Amazon RDS. Explain **at least three** important decisions that lead to this physical architecture. **(7 marks)**

> **(b)** 根据你在 (a) 改正后的图，用下列服务中的**一部分**画出 TixNow 在 AWS 上的**物理详细部署图**：Amazon API Gateway、Amazon CloudFront、Amazon S3、AWS WAF、Amazon Cognito、Amazon EKS、AWS Lambda、Amazon SQS、

github.com/bibi45c · 免费分享，禁止售卖

> Amazon DynamoDB、Amazon ElastiCache、Amazon RDS。解释导出这个物理架构的**至少三个**重要决策。 **(7 分)**

Section C (12 marks) · C 部分（12 分）

Question 3 · 第 3 题

(a) The existing box-office system **VenueBox** (Section 6) must be moved to the cloud within 6 months, before the National Games, and later integrated with TixNow. Recommend a **migration strategy** for VenueBox, justify it, and state **one** risk of your choice. **(4 marks)**

> **(a)** 现有的售票柜台系统 **VenueBox**（第 6 节）必须在 6 个月内、全国运动会之前搬到云上，之后再和 TixNow 集成。为 VenueBox 推荐一个**迁移策略**，论证它，并说出你这个选择的**一个**风险。 **(4 分)**

(b) Refer to the seat-booking fragment of the domain model in Figure 5. During the first on-sale, **two fans were able to hold the same seat** at the same time. **Identify** the most likely cause, **propose** a countermeasure by changing the domain model, and state the **main drawback** of your countermeasure. **(4 marks)**

> **(b)** 参考图 5 领域模型中的订座片段。第一次开售时，**两名观众同时保留了同一个座位。指出**最可能的原因，**提出**通过修改领域模型的对策，并说明对策的**主要缺点。 (4 分)**

(c) During on-sale peaks the external Payment Gateway sometimes takes more than 20 seconds to answer or times out. Payment requests then block the threads of the payment service, other requests slow down, and many seat holds expire before payment completes. The 8-minute hold rule in Section 3 **must not be changed**. Propose **two** resilience measures and explain how each one helps. **(4 marks)**

> **(c)** 开售高峰时，外部支付网关有时超过 20 秒才回应，或者直接超时。支付请求于是堵住了支付服务的线程，其他请求也变慢，很多座位保留在付款完成前就过期了。第 3 节的 8 分钟保留规则**不能改。**提出**两个**韧性措施，并解释每个措施怎样起作用。 **(4 分)**

— END OF PAPER · 试卷结束 —

github.com/bibi45c · 免费分享，禁止售卖

Appendix A — Case Study: TixNow National Ticketing Platform · 附录 A 案例：TixNow 全国票务平台

1. Background · 背景

The national arts and sports agency is building **TixNow**, a ticketing platform shared by venues (concert halls, stadiums, theatres), **event organisers** and **fans**. The platform runs four applications:

> 全国艺术与体育管理局正在建设 **TixNow**，一个由场馆（音乐厅、体育场、剧院）、**活动主办方**和**观众**共用的票务平台。平台运行四个应用：

- **TixNow Concerts** and **TixNow Sports** — fans browse events, join the on-sale queue, select seats and pay;
  TixNow Concerts 和 **TixNow Sports**：观众浏览活动、加入开售排队、选座、付款；
- **TixNow Resale** — fans resell tickets to other fans at face value;
  TixNow Resale：观众按票面价把票转售给其他观众；
- **TixNow Venue** — venue staff scan tickets at the gates with handheld scanners.
  TixNow Venue：场馆工作人员在闸口用手持扫描器验票。

Roles: **Fan**, **Organiser Admin**, **Venue Staff**, **Finance Officer**, **Platform Admin**. Organisers and venues are **tenants** of the platform. External systems: **Payment Gateway**, **SMS/Email Provider**, the national **Singpass**-style identity service used to verify fans for high-demand events.

> 角色：**观众**、**主办方管理员**、**场馆工作人员**、**财务专员**、**平台管理员**。主办方和场馆是平台的**租户**。外部系统：**支付网关**、**短信 / 邮件服务商**，以及类似 **Singpass** 的国家身份认证服务（热门活动用它核验观众身份）。

2. Extensibility · 可扩展性

The platform shall provide common services and libraries that can be shared across the applications and exposed to external applications. In the near future it shall support **timed-entry tickets for theatres and museums** and the sale of **event merchandise**.

> 平台应提供能在各应用之间共享、并开放给外部应用使用的公共服务和库。近期还要支持**剧院和博物馆的分时段入场票**，以及**活动周边商品**的销售。

3. Performance, Scalability and Multitenancy · 性能、可扩展性与多租户

- For a popular concert, **500,000 fans** may join the virtual queue in the first **10 minutes** of the on-sale; the platform shall handle up to **50,000 seat-selection requests per minute**.
  热门演唱会开售的前 **10 分钟**内，可能有 **50 万观众**加入虚拟排队；平台要能处理每分钟最多 **5 万次选座请求。**
- Interactive requests (browse events, select seats, view my tickets) shall complete within **2 seconds for 90%** of the requests.
  交互请求（浏览活动、选座、查看我的票）**90% 要在 2 秒内**完成。
- A seat hold expires **8 minutes** after it is created if the fan has not paid.
  座位保留创建后，如果观众没付款，**8 分钟**后失效。
- At a stadium, **60,000 fans** enter within **90 minutes**; each scan shall be validated in **less than 1 second**, even when the venue network is unstable.
  体育场里 **6 万名观众**要在 **90 分钟**内入场；即使场馆网络不稳定，每次扫描也要在 **1 秒内**完成校验。
- The organiser sales dashboard shall be updated **every minute**.
  主办方销售看板**每分钟**更新一次。
- The platform **must support multitenancy**: the sales and customer data of each organiser and of each venue must be isolated from other tenants. Data may be partitioned by organiser and by event.
  平台**必须支持多租户**：每个主办方和每个场馆的销售与客户数据都要和其他租户隔离。数据可以按主办方和按活动分区。

4. Security · 安全

The platform follows the DMZ / reverse-proxy practice. Personal data is encrypted and handled according to the PDPA. Payment card data is handled only by the Payment Gateway and by as few platform subsystems as possible (PCI-DSS). Rate limiting and bot detection protect the on-sale queue.

github.com/bibi45c · 免费分享，禁止售卖

平台遵循 DMZ / 反向代理的做法。个人数据加密，并按新加坡个人数据保护法（PDPA）处理。支付卡数据只由支付网关和尽量少的平台子系统处理（PCI-DSS）。开售排队由限流和机器人检测保护。

5. Use Cases · 用例

ID	Use case 用例	Description 说明
UC1	Browse Events 浏览活动	Fans list events by category and date and view the event details and the venue seat map. Concerts, matches and theatre shows have different attributes (e.g. artists and setlist; teams and league; cast and running time). 观众按类别和日期列出活动，查看活动详情和场馆座位图。演唱会、比赛、话剧的字段不同（如歌手和歌单；球队和联赛；演员和时长）。
UC2	Join Queue and Select Seats 排队并选座	Fans join a virtual queue for an event. When admitted, a fan selects up to 4 seats in one or more price categories. The selected seats are **held** for the fan for 8 minutes. 观众加入某个活动的虚拟排队。被放行后，观众在一个或多个价位里最多选 4 个座位。选中的座位为该观众**保留** 8 分钟。
UC3	Pay and Issue Tickets 付款并出票	The fan pays for the held seats through the Payment Gateway. For each seat an e-ticket with a QR code is issued to the fan's ticket wallet, and a confirmation is sent by SMS/email. 观众通过支付网关为保留的座位付款。每个座位出一张带二维码的电子票，放进观众的票夹，并通过短信 / 邮件发送确认。
UC4	Resell Ticket 转售门票	A fan lists a ticket for resale at face value. When another fan buys it, the platform pays the seller, invalidates the old QR code and issues a new e-ticket to the buyer. 观众按票面价挂出一张票转售。另一位观众买下后，平台付款给卖家，作废旧二维码，并给买家出一张新电子票。
UC5	Scan Ticket at Gate 闸口验票	Venue staff scan the QR code with a handheld scanner. The scanner shows "valid" or "invalid" and records the entry. 场馆工作人员用手持扫描器扫二维码。扫描器显示"有效"或"无效"，并记录入场。
UC6	Manage Event 管理活动	The Organiser Admin creates an event at a venue, defines price categories, assigns seats of the venue's seat map to the categories and sets the on-sale time. 主办方管理员在某个场馆创建活动，定义价位，把场馆座位图上的座位分配到各价位，并设置开售时间。
UC7	View Sales Dashboard 查看销售看板	The Organiser Admin views tickets sold and revenue per price category, updated every minute. 主办方管理员查看每个价位的售票数和收入，每分钟更新。
UC8	Settle with Organiser 与主办方结算	Seven days after each event, the Finance Officer runs the settlement that pays the organiser (batch). 每场活动结束 7 天后，财务专员运行结算，给主办方付款（批处理）。

6. Existing System: VenueBox · 现有系统 VenueBox

VenueBox is the current box-office system used at venue counters. It is an 8-year-old Java monolith running on two physical servers with an Oracle database in the agency's data centre. It works reliably but cannot scale for online on-sales, and the hardware support contract ends in 6 months.

VenueBox 是目前场馆售票柜台用的系统。它是一个用了 8 年的 Java 单体应用，跑在管理局数据中心的两台物理服务器上，用 Oracle 数据库。它运行稳定，但无法扩展到线上开售的规模，而且硬件维保合同 6 个月后到期。

7. Domain Model — Seat-Booking Fragment (Figure 5) · 领域模型：订座片段（图 5）

<>	Entities 实体	Notes 说明
EventManager	Event, PriceCategory, **Seat** (status: available / held / sold) 活动、价位、**座位**（状态：可售 / 保留 / 已售）	Seat is part of the Event aggregate. Seat 属于 Event 聚合。

github.com/bibi45c · 免费分享，禁止售卖

<>	**Entities** 实体	**Notes** 说明
BookingManager	Booking, **SeatHold** (seatId, fanId, expiresAt), **SeatReplica** (read-only copy of Seat) 订单、**座位保留**、**座位副本**（Seat 的只读副本）	SeatReplica is updated by SeatStatusChanged events published by EventManager. SeatReplica 由 EventManager 发布的 SeatStatusChanged 事件更新。

Business rule: **a seat can be held or sold to only one fan at a time.** When a fan selects a seat, BookingManager checks the seat status in SeatReplica and, if it is available, creates a SeatHold and publishes a SeatHeld event; EventManager then sets the Seat to held.

> 业务规则：**一个座位同一时间只能被一名观众保留或买下。**观众选座时，BookingManager 在 SeatReplica 里查座位状态，如果可售就创建一个 SeatHold，并发布 SeatHeld 事件；EventManager 收到后再把 Seat 设为"保留"。

github.com/bibi45c · 免费分享，禁止售卖

模拟卷 1·2 参考答案与翻页指引

先自己限时做，再看这里。每题给：题型 → 应该翻到手册哪几页 → 得分点 → 参考答案要点

页码按当前版《SWE5001 考场答题手册》自动生成；手册改版后重跑 build_mock.py 会同步更新。
这是练习卷，得分点按样卷评分方式推断，不是官方答案；合理的其他答案也能得分。

用法：练翻书比练答案更重要

步	做什么
1	拿到题先看**指令词和分值**，用手册"题型速认表"（p.5）判断题型，记下你翻到的页
2	限时：每 1 分 3 分钟，2 分题不超过 7 分钟
3	做完对照下面的"应翻页"：**你翻到的页对不对、翻了多久**；翻不到的地方就在手册上贴标签或写批注
4	再对得分点；写不出来的英文句子，去 PART 3 句型（p.58）找对应功能的句子

模拟卷 1 · GreenCharge 充电平台

Q1(a)(i) 平均响应时间当通过标准 · 2 分 · T11

应翻页： T11 负载测试 p.38，指标定义 p.67

得分点	要点
原因（1）	不同意。需求是 **90% < 2 秒**（百分位），平均值会掩盖慢请求；而且 10 分钟测试没有**工作负载模型**（曲线、并发数、场景组合）
办法（1）	写工作负载模型：晚高峰逐步加到 **20,000 并发司机**，场景组合（如 50% 找桩、20% 预约、15% 开始、15% 结束），用 JMeter 从多台施压机执行；通过标准 **P90 < 2 s**、错误率 < 0.1%

> *I disagree. Section 4 requires 90% of requests within 2 seconds, but an average hides slow requests. Pass criterion: P90 < 2 s with a defined workload model (ramp up to 20,000 concurrent drivers, a fixed request mix), run with JMeter from several load generators.*

Q1(a)(ii) 队列还是事件流 · 2 分 · T10

应翻页： T10 p.35

1. **多个独立消费者：** 看板（每 5 分钟）和月度结算要读同一批读数；事件流让两个消费者组各读一遍，消息队列一条消息只给一个消费者、投递后就删。
2. **保留与重放：** 月结要读一整个月的读数，出错或有争议时还要重跑；事件流按保留期保存、可从头重放（保留期要设到一个月以上：Kinesis 最长 365 天、Kafka 可配置，或同时落到 S3 给批处理）。
3. （补充）**顺序：** 按 sessionId 分区能保证同一会话的读数有序；但读数是累计值，乱序时取最新时间戳也能处理，所以顺序不是最强的理由。

Q1(b)(i) StationManager 内存高 · 5 分 · T1

应翻页： 五类症状 p.16，句子 p.16

得分点	要点
原因（2）	ChargingStation 聚合里挂着**所有会话的 MeterReading**（每个充电中的桩每 30 秒一条），加载一个站就把整天的读数全载入内存，所以内存随一天增长

github.com/bibi45c · 免费分享，禁止售卖

得分点	要点
对策（2）	按"设计小聚合"把 MeterReading 拆出成独立聚合（可以留在 StationManager，也可以移到 SessionManager 按会话管理），**只用 ID 引用**（connectorId / sessionId），按需加载
缺点（1）	要看某站的读数得多查一次；跨聚合更新只能最终一致

分值拆法是推断（PAST 5 分题是原因 2 + 对策 1.5 + 1.5 + 缺点 0）。**同样可得分的方向：** 把 MeterReading 移到独立的计量服务（如 MeteringManager + Amazon Timestream）；指出 ChargerStatus 的高频写入也在同一个大聚合里，造成锁竞争。

Q1(b)(ii) 可用状态不及时 · 4 分 · T1

应翻页： 五类症状 p.16，搬到一起还是副本 p.16

得分点	要点
原因（1.5）	ChargerStatus 在 StationManager，ConnectorAvailability / AvailabilityMap 在 AvailabilityManager，后者由前者派生；每次状态变化都要**跨进程**同步到另一个服务，比进程内慢得多（加上 StationManager 的大聚合加载更慢）。**另一个同样成立的原因：** [A] AvailabilityMap 把全市的 ConnectorAvailability 放在一个聚合里，更新一个桩就要加载、锁住整张地图，对策是按桩或按区域拆成小聚合
对策（1.5）	把 ChargerStatus 移进 AvailabilityManager，和 ConnectorAvailability 放一起，状态一到就在同一个服务里更新地图；站点配置（Station、Connector、Tariff）留在 StationManager。副本方案也可以写，但要求 10 秒内且写入频繁，**搬到一起更稳**
缺点（1）	AvailabilityManager 变大、要承受全部状态写入；StationManager 需要状态时要通过 API 或事件读取（最终一致）

Q1(c) 充电桩上报 API 延迟 · 6 分 · T3

应翻页： 性能判断链 p.21，PAST2 Q1(c) 范文 p.49

得分点	要点
原因（2）	① **同步**：回复前重算全市可用地图，15,000 台并发排队；② **PUT 同一个共享资源** `/chargers/status`，并发互相覆盖、阻塞，而每次上报其实是一个新事件；③ 输入冗余 / 混杂：tariff、driverId 服务器已知或无关，状态和电表读数混在一起
重构（2）	面向资源 + 异步：状态 `POST /connectors/{connectorId}/status-events`，读数 `POST /sessions/{sessionId}/meter-readings`；校验后发布到按 **connectorId** 分区的事件流（Kinesis / MSK），立即返回 **202**；AvailabilityManager 并行按序消费，只更新受影响的地图区块，10 秒内推送
例子（2）	`POST /connectors/C-1042/status-events` 体 `{"status":"Occupied","timestamp":"18:02:10"}` → **202 Accepted**；`POST /sessions/S-88123/meter-readings` 体 `{"kWh":12.4,"timestamp":"18:02:30"}` → 202

Q1(d)(e) 两个质量属性 + 两个云服务 · 2 + 4 分 · T7 + T6

应翻页： T6 选存储 p.27，PAST2 范文 p.50，换算 p.68

- **(d)** 写入可扩展性（15,000 台，充电时每 30 秒一条 → 最多约 **500 次 / 秒**，还有状态变化）+ 低延迟（10 秒内可见、交互 90% < 2 秒）。比复杂查询重要：数据简单、按桩 / 会话 ID 读写。
- **(e) Amazon DynamoDB**（按 connectorId / sessionId 分区，写吞吐水平扩展，毫秒级）存读数和状态；**ElastiCache for Redis** 存每个桩的最新可用状态，地图查询亚毫秒。也可：Amazon Timestream 存读数（时序），S3 存历史给结算。每个服务都要写**满足哪个属性、靠什么机制。**

Q2(a) 混合云原生架构 · 8 分 · T5

应翻页： T5 p.23，K8s 计划 p.24，PAST Q2(a) 范文 p.46

github.com/bibi45c · 免费分享，禁止售卖

小问	要点
(i) 组件（3）	App / 门户 → CloudFront + API Gateway（+ Cognito）→ EKS 微服务（Driver、Station、Session、Availability、Payment、Billing）；**充电桩接入：AWS IoT Core / Kinesis**；事件流（MSK / Kinesis）；Redis；RDS（会话、支付）+ DynamoDB（读数）；Lambda + SNS 发通知；定时任务做月结算；服务注册、服务网格、ECR、CI/CD、监控
(ii) 通信（2）	开始充电要立刻授权 → 同步 REST / gRPC（Session → Driver、Payment）；状态和读数 → 事件流（Availability、Billing、看板各自消费）；通知 → 队列 / Lambda。**在图上标出**
(iii) 状态（1）	AvailabilityManager 保存每个桩的最新状态 → 外部状态存储 ElastiCache Redis（或 SessionManager 的进行中会话放 Redis + RDS）
(iv) K8s（2）	namespace；每服务 **Deployment** ≥ 3 个 **Pod** 跨可用区 + ClusterIP **Service**；Ingress 在 API Gateway 后；HPA 给 Availability、Session（按 CPU 或事件流积压）；月结算 **CronJob**；Kafka / Redis / 数据库用托管服务；ConfigMap / Secret

Q2(b) LLM 行程助手 · 3 分 · 超纲预备卡 A

应翻页： 预备卡 A p.9，T17 熔断 p.42

- **异步：** `POST /assistant/questions` 立即返回 **202 + questionId**，App 不被阻塞（满足 §4：90% 在 2 秒内确认收到）；后台 worker 调 LLM，结果用推送 / WebSocket 返回（或流式分段显示）。
- **超时 + 重试 + 熔断降级：** 设超时（约 6–8 秒）、最多重试一次，保证 15 秒内给出答案；失败率高时熔断，返回缓存的常见路线或规则算法给出的充电站点。
- **缓存与集成层：** 常见问题结果缓存在 Redis；LLM 调用放 External Integration 层；单独监控 P95 和失败率。

Q2(c) 共置 · 2 分 · T12

应翻页： T12 p.36

StationManager + AvailabilityManager：每一次桩状态变化（15,000 台）都要从 ChargerStatus 传到 ConnectorAvailability，通信亲和度最高；共置后变进程内调用，延迟低，更容易满足 10 秒；两者负载都由桩上报驱动，可一起扩缩。**不要**把 PaymentManager 和别的服务放一起（PCI-DSS）。

Q3(a) 打包部署找错 · 6 分 · T4

应翻页： T4 清单 p.13，句子 p.13

任选两个，每个"识别 + 纠正 + 论证"：

识别	纠正	论证
Payment Manager Control **部署到手机**	删除这条 deploy，留在后端	支付由后端经 Payment Gateway 完成；PCI-DSS 要求尽量少的子系统接触支付
Session Manager Data Access **没部署到手机**	加 deploy 到 Mobile Device with Local Storage	UC3 要求收据存在手机、可离线查看（Receipt 在这个子系统里）
MeterReading 打包进 Payment Manager Data Access	改打包进 **Station Manager Data Access**	领域模型中 MeterReading 属于 StationManager
PaymentGatewayAdapter 打包进 Session Manager Control	新建 **Payment Manager External Integration**，部署在后端内部区	对接外部系统的 boundary 放集成层；它属于支付上下文；外部变化只影响集成层

Q3(b) "每站每日会话数"当指标 · 4 分 · T8

应翻页： T8 p.33，PAST Q1(e) 范文 p.45　**分值（推断）：** 表态并论证 1 + 两个指标各约 1.5

- **不同意：** 会话数只是**使用量**，取决于站点位置和车流；故障的桩不产生会话但数字看不出原因，也反映不了司机等了多久、启动失败几次。
- **更好的指标 1（可用性）：** 充电启动成功率（Yield）= 成功启动次数 / 启动尝试次数；或桩可用时间占比 = 非故障时间 / 总时间。
- **更好的指标 2（体验）：** 扫码到开始充电的平均（或 P90）耗时；或启动失败率。每个指标都要带理由。

github.com/bibi45c · 免费分享，禁止售卖

Q3(c) 结束充电路径的可用性 · 2 分 · T13

应翻页： T13 p.40，几个 9 表 p.68

- 0.9995 × 0.999 × 0.999 × 0.995 ≈ **99.25%** < 99.5%，**不满足。**
- 改进（任一）：最弱的是外部 Payment Gateway（99.5%）→ 接**第二家支付网关做故障切换**：1 − 0.005² = 99.9975%，整体 ≈ **99.75%**；或把扣款从结束请求里拿出来：结束时先记录会话、出收据，扣款**异步**进行（带重试和幂等键），同步路径只剩 0.9995 × 0.999 × 0.999 ≈ 99.75%（前提：业务接受稍后扣款）。

模拟卷 2 · TixNow 票务平台

Q1(a) 最关键的架构方面 · 2 分 · T9

应翻页： T9 p.31，PAST3 Q1(a) 范文 p.54

种子：票和座位；生产者：主办方、场馆（转售时粉丝也是生产者）；消费者：粉丝。最关键（任选一个并引用数字）：**开售交互的可扩展性与公平性**：10 分钟内 50 万人进入虚拟队列、每分钟 5 万次选座，要用队列削峰、弹性扩展，并保证一个座位只卖一次；或**多租户 + 共享服务**（主办方数据隔离、共享目录 / 票务 / 支付 API）。

Q1(b)(i) 类图 · 5 分 · T2

应翻页： T2 p.18，PAST3 Q1(b) 范文 p.54

服务	实体（2–3 个属性）
UserManager	Fan、OrganiserAdmin、VenueStaff（← User）；Organiser（主办方机构，租户）
VenueManager	Venue、Seat（row, number）
EventManager	Event（date, category）、PriceCategory（name, price）
QueueManager	QueueEntry（position, admittedAt）
BookingManager	**SeatInventory**（eventId, seatId, status：available / held / sold）与 SeatHold（expiresAt）同一个小聚合；Order（status）、OrderItem
PaymentManager	Payment（amount, status）— PCI-DSS 只在这里
TicketManager	Ticket（ticketNo, qrSignature, status）、Wallet
ResaleManager	ResaleListing（price = face value, status）

关系：Organiser 1— *Event；Venue 1*— Seat；Venue 1— *Event；Event 1*— SeatInventory（按 ID 引用）；SeatInventory *—1 Seat；SeatInventory* —1 PriceCategory；SeatInventory ◆ SeatHold（0..1）；Fan 1— *Order；Order ◆ OrderItem（1..4）；OrderItem 1—1 Ticket；Ticket* —1 SeatInventory；Order 1—1 Payment；Ticket 1—0..1 ResaleListing。**删掉：** SMS / Email Provider、Payment Gateway（外部系统），QR 码（Ticket 的属性），日期、数量（属性）。座位状态和保留放在同一个小聚合，是为了守住"一座一人"的规则（也就是 Q3(b) 的纠正）；题干要求不参考 Figure 5。**分值（推断）：** 实体与服务分组 2 + 关系与多重性 2 + 属性 1。

Q1(b)(ii) 共享服务 · 4 分 · T2

用户与身份（四个应用 + 实名核验）；活动目录（Concerts、Sports、Resale + 剧场扩展）；场馆与座位图（Concerts、Sports、Venue + 剧场）；虚拟排队（开售、周边限量发售）；订座 / 座位保留；支付（全部，PCI-DSS）；出票与票夹（Concerts、Sports、Resale、Venue 验票、分时段入场）；通知；转售（Concerts、Sports 都要展示和购买转售票，所以算共享；若只给 Resale 用就归应用专属）；销售统计与结算。**每个写给谁用、做什么。**

Q1(c) 开放哪两个 API + Level 3 · 2 + 4 分 · T9 + T3

应翻页： T3 Level 3 p.21，PAST3 Q1(c) 范文 p.55

- **(i)** 例：**活动目录 API → 媒体和旅游合作网站**（消费者）：展示活动并跳转购票，带来更多粉丝；**活动管理 API → 主办方自己的系统**（生产者）：直接创建活动、价位，不用在门户手工录入，带来更多种子。
- **(ii)** 例：活动管理 API（给主办方系统）

github.com/bibi45c · 免费分享，禁止售卖

URI	HTTP	输入	输出（links）	用途
`/events?organiserId=..&status=..`	GET	查询参数	200；列表；links: self, next, 每个活动	列出主办方的活动
`/events/{id}`	GET	—	200；详情；**草稿**时 links: self, update-event, add-price-category, publish；**开售后**只剩 self, sales	看一个活动
`/events`	POST	name, venueId, date, category	**201**；links: self, add-price-category	创建活动（草稿）
`/events/{id}`	PATCH	要改的字段	200；links: self, publish；开售后改价位返回 **409**	修改活动
`/events/{id}/price-categories`	POST	name, price, seats	201；links: self, event	添加价位

收尾两句：复数名词 + 正确动词和状态码 + 每个响应带随状态变化的链接 = Level 3； `Accept-version` 头做版本，OAuth 2.0 按主办方租户保护。

Q1(d) 四种存储 · 4 分 · 超纲预备卡 D

应翻页： 预备卡 D p.10，T6 选存储 p.27

数据	存储类型 → 产品	理由
活动目录	文档库 → Amazon DocumentDB（或 DynamoDB）	不同类别字段不同，schema 灵活
座位保留	内存键值 → **ElastiCache for Redis**（键带 8 分钟 TTL）	高并发写、毫秒级、到期自动失效；（DynamoDB 的 TTL 删除不及时，若用它要在代码里检查 expiresAt）
订单与支付	关系库 → **Amazon RDS / Aurora（Multi-AZ）**	订单和明细要在一个 ACID 事务里保存，强一致
看板计数	内存计数 → ElastiCache Redis（INCR）或 DynamoDB 原子计数	每分钟刷新、读写快；历史可落 Redshift / S3

Q1(e) 离线验票 · 4 分 · 新题（推理题）

应翻页： 四步下笔 p.8，T13 可用性战术 p.40

- **(i) 设计（2）：** 开闸前把本场、本闸口的**有效票清单 / 二维码签名**下载到扫描器本地存储；二维码**数字签名**，本地验签 + 查本地清单，不依赖网络，< 1 秒；入场记录先存本地，网络恢复后**异步同步**到后端；后端定期推送增量（转售作废的票）。
- **(ii) 风险（2）：** 同一张票在两个离线闸口被重复使用，或转售后旧二维码在本地仍有效。**缓解：** 高频同步作废清单；用会**定时轮换**的二维码；同一场馆的扫描器在局域网内互相同步入场记录；按看台分配闸口，一张票只能从指定闸口进；事后对重复记录稽核。

Q2(a) 逻辑详细部署图找三个错 · 6 分 · T4

应翻页： T4 清单（四个维度）p.13，PAST3 Q2(a) 范文 p.56

识别	纠正	论证
Sports **没有 Control** 子系统	加 Sports Control，部署到 Application 节点	分层要求 Presentation → Control，否则 Sports 界面直接调平台服务或借用 Concerts 控制层，两个应用耦合
共享的 Seat Map Service 在 Application 节点，只给 Concerts 用的推荐服务在 Platform 节点	两者对调：座位图放 Platform；推荐放进 Concerts 自己的子系统	Platform 只放多个应用复用的服务；剧场扩展要复用座位图
Payment Service **直接调外部支付网关**，且**没有 DMZ**	加 DMZ，只放反向代理 / API 入口；外部调用经 Integration 子系统，Integration 节点部署在**内部区**，经出站代理 / NAT 访问支付网关	§4 要求 DMZ 反向代理；PCI-DSS 要求支付尽量少接触；集成层隔离外部变化

github.com/bibi45c · 免费分享，禁止售卖

识别	纠正	论证
一个共用的 Data Subsystem 给所有服务和主办方	每服务一个数据存储，按主办方分区 / 租户密钥	§3 要求多租户隔离、按主办方分区

Q2(b) AWS 物理部署 · 7 分 · T6

应翻页： T6 逻辑节点 → AWS p.27，PAST3 Q2(b) 范文 p.57　**分值（推断）：** 图约 3 + 决策约 4

图：CloudFront + S3（前端、座位图图片）→ **WAF + API Gateway + Cognito**（VPC 外的托管边缘入口，起 DMZ 作用；限流防机器人）→ VPC 两个可用区私有子网：**EKS**（活动、订座、出票、转售服务，HPA）、**Lambda**（出票后通知、看板聚合）、**SQS**（后端缓冲选座 / 下单请求）→ 数据：**ElastiCache**（座位保留 TTL、可用座位缓存、计数）、**DynamoDB**（排队记录、电子票）、**RDS Multi-AZ**（订单、支付、结算）、S3；集成子网经 NAT 访问支付网关和短信。

决策（至少三个，每个"选了什么 + 为什么 + 满足哪个需求"）：① 边缘用 WAF + API Gateway 限流 + Cognito，挡机器人、统一入口（§4）；② **虚拟等候室**：排队令牌和位置存在 ElastiCache 有序集合（或 DynamoDB），按可控速率发放入场令牌，吸收 10 分钟 50 万人的峰值；后端用 SQS 缓冲选座 / 下单请求，按 5 万 / 分钟扩容；③ 常驻服务用 EKS、事件驱动用 Lambda；④ 多语言持久化 + 按主办方分区满足多租户（§3）。

Q3(a) VenueBox 迁移策略 · 4 分 · 超纲预备卡 C

应翻页： 预备卡 C p.10，T17 迁移 p.42　**分值（推断）：** 策略 1 + 理由 2 + 风险 1

推荐 **Replatform**（或 Rehost）：6 个月内硬件合同到期、国运会前不能出事 → 不做重构。把 Java 单体容器化跑在 EKS / ECS（或先 rehost 到 EC2），数据库迁到 **Amazon RDS for Oracle**（托管备份、多可用区）；之后再用"逐步替换"的方式拆进 TixNow 服务。**理由：** 风险和成本低、见效快、柜台业务不中断；Rearchitect 6 个月做不完。**风险：** Oracle 许可证和兼容性、柜台到云的网络延迟；缓解：先在预发布环境演练、保留回滚方案、用 VPN / 专线。

Q3(b) 两个粉丝保留了同一个座位 · 4 分 · T1（数据不一致）

应翻页： 五类症状 p.16，副本判断 p.16

得分点	要点
原因（1.5）	"一个座位同时只能给一个粉丝"要求 Seat 状态和 SeatHold **一起**变，但它们分在两个服务；BookingManager 查的是**只读副本** SeatReplica，靠事件**最终一致**地更新，副本还没更新时第二个粉丝也通过了检查
对策（1.5）	把 SeatHold 和座位状态放进**同一个小聚合**（如 BookingManager 里每个场次座位一个 SeatInventory 聚合，或把 SeatHold 移进 EventManager 的座位聚合），由聚合根在一个事务里检查并设为 held（乐观锁），不再用副本检查；Event 和座位拆开，避免整场活动成一个大聚合
缺点（1）	热门座位 / 区域并发冲突增多，部分粉丝收到"已被选"要重选或重试；服务边界变化

Q3(c) 支付网关慢 · 4 分 · T17

应翻页： T17 p.42，预备卡 A p.9

任选两个，每个"措施 + 怎么帮上忙"（各 2 分）：

1. **超时 + 熔断：** 支付调用设 5 秒左右超时；失败率超过阈值时熔断、快速失败，返回"支付处理中 / 请稍后"，线程不再被堵住，防止级联故障。
2. **舱壁：** 支付调用用单独的线程池 / 实例，它慢了也不拖垮选座和浏览。
3. **异步支付 + 幂等键：** 收到支付请求先回 202，经队列调用网关并重试；用**幂等键**防止重复扣款；题目规定 8 分钟规则不能改，所以要在保留期内尽早发起支付，剩余时间不足时提示粉丝（不能延长保留）。
4. **指数退避重试：** 50 / 100 / 200 ms 递增，避免重试风暴。

github.com/bibi45c · 免费分享，禁止售卖

模拟卷讲解补充：失分点 · 答题技巧 · 图例

手册页码以本次重建后的《考场答题手册》为准：T4 p.13，T2 p.18，T3 Level 3 p.21，T9 p.31。

一、模拟卷 1 GreenCharge：薄弱环节（约 28 / 50）

#	薄弱点	证据	下次这样做
1	**T4 打包部署找错**	Q3(a) 6 分只拿 1–1.5	先按 §7 核对每个实体属于哪个服务，再逐条查 deploy 线，最后看 boundary 是否在 External Integration 层
2	读领域模型的跨框连线	Q1(b)(ii) 服务认错	跨服务关联就是题眼，原因里点名这两个服务
3	T5 架构图 + 异步设计	Q2(a)(i)–(iii)、Q2(b) 共 9 分只拿 1.5	照 T5 组件表画框；(ii) 写同步 + 异步两句；(iii) 写外部状态存储 Redis；LLM 题写 202 + 后台调用 + 超时重试熔断 + 推送
4	没锚定案例事实	Q2(c) 说"每次充电都结算"，附录写的是月度批处理	理由只用附录原文
5	可用性计算与外部依赖	Q3(c) 没算出 99.25%	一定写出数值；外部依赖改成第二家并联切换，或异步移出同步路径
6	质量属性用词	把完整性、故障转移都叫 Security	准确性 = Integrity，故障转移 = Availability
7	英文作答	几乎全中文	用 *Cause / Countermeasure / Drawback* 标签 + 短句 + 案例数字

二、模拟卷 2 TixNow：Section A 讲解

Q1(a) 最关键的架构方面（2 分）

三句话拿满： 种子 + 双方 + 交互写一句 → 写**一个**方面并带上 §3 的数字 → 加一句"做不到会怎样"。常见失分点：写了两个方面（可扩展性 + 多租户），还套用了 PAST2 的"查看服务从缓存读最新值"。

The seeds of the platform are event tickets and seats, produced by organisers and venues and consumed by fans through browsing events, joining the on-sale queue, selecting seats and paying; in resale, fans also act as producers.

> 平台的种子是活动门票和座位，由主办方和场馆产生，粉丝通过浏览活动、加入开售队列、选座和付款来使用；转售时粉丝也是生产者。

The most crucial architectural aspect is the scalability of the on-sale services: 500,000 fans may join the virtual queue in the first 10 minutes and the platform must handle 50,000 seat-selection requests per minute, so the services must be stateless and scale out automatically, keeping 90% of requests within 2 seconds.

> 最关键的架构方面是开售服务的可扩展性：开售头 10 分钟可能有 50 万粉丝进入虚拟队列，平台每分钟要处理 5 万次选座请求，所以服务必须无状态、能自动扩容，保持 90% 的请求在 2 秒内完成。

The virtual queue absorbs the peak and admits fans at a rate the seat service can handle, and each seat must be held or sold to only one fan, so the system stays fair. If the on-sale fails or sells a seat twice, fans and organisers lose trust and leave the platform.

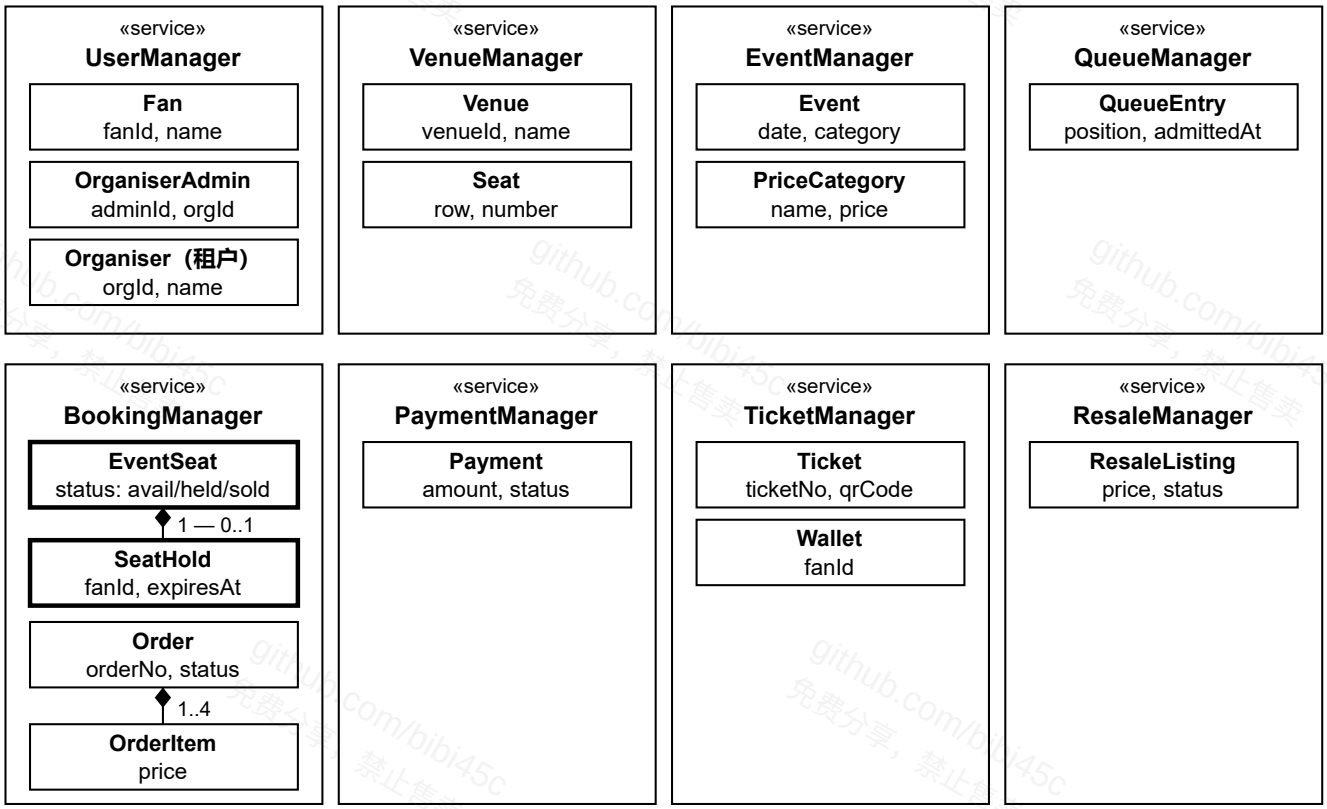
> 虚拟队列吸收洪峰，按选座服务能承受的速度放人进来；每个座位只能被一个粉丝占座或买下，保证公平。如果开售崩溃或一座两卖，粉丝和主办方会失去信任、离开平台。

Q1(b)(i) 类图（5 分）：只过题目点名的 UC2、3、4、6

考场快速做法（约 8 分钟）： ① 每条用例圈 2–3 个名词；② 三条删除规则：外部系统删（SMS、Payment Gateway），属性删（QR 码、8 分钟、面值），同一个人的不同角色合并（买家、卖家都是 Fan，在关联线上标 seller / buyer）；③ 分进服务框，每个实体写 2 个属性，连 6–8 条线并标多重性。分值（推断）：实体与分组 2 + 关系与多重性 2 + 属性 1。

最值钱的一点： 已售 / 已保留的状态记在 **EventSeat**（某场活动的某个座位），不记在 Seat 上。Seat 是场馆里固定的位子，周六卖了周日还要卖。EventSeat 和 SeatHold 放在同一个小聚合里，才能守住"一座一人"；这也是 Q3(b) 的答案方向。

github.com/bibi45c · 免费分享，禁止售卖



粗框 = EventSeat 与 SeatHold 同一个小聚合（守住"一座一人"）；◆ = 组合。跨服务的关系按 ID 引用，见下表。

关系（多重性）	关系（多重性）
Organiser 1 — * Event	Fan 1 — * Order
Venue 1 — * Event；Venue 1 — * Seat	Order ◆ OrderItem（1..4）
Event 1 — * EventSeat	OrderItem 1 — 1 Ticket
EventSeat * — 1 Seat；EventSeat * — 1 PriceCategory	Order 1 — 1 Payment
EventSeat ◆ SeatHold（0..1）	Ticket 1 — 0..1 ResaleListing（seller / buyer 都是 Fan）

Q1(b)(ii) 与 Q1(c)(i)：两题的"给谁"不一样

	(b)(ii) 共享服务（4 分）	(c)(i) 开放 API（2 分）
问什么	平台**内部**被多个应用共用的服务	从上面挑 **2 个**开放给**外部**
"给谁"指	四个应用：Concerts、Sports、Resale、Venue	主办方自己的系统、合作网站等
写几个	4–5 个，每个写"给谁用、做什么"	正好 2 个，生产者、消费者各一个最稳

(b)(ii) 检查办法： 每一行都问"点名了几个应用？"只点名一个的不算共享（推荐服务只给 Concerts 用）。

共享服务	给谁用	做什么
Ticket Service	Concerts、Sports 出票；Resale 作废重发；Venue 验票；以后分时段票	电子票、票夹、QR 码、验票
Payment Service	Concerts、Sports、Resale；以后周边	对接 Payment Gateway 付款退款；PCI-DSS 集中
Event Catalogue Service	Concerts、Sports、Resale；以后剧场	活动、价格档、不同类型属性
Virtual Queue Service	Concerts、Sports；以后限量周边	排队、按速度放人、限流防机器人（§4）
Venue & Seat Map Service	Concerts、Sports、Venue；以后剧场	场馆和座位图

github.com/bibi45c · 免费分享，禁止售卖

Ticket Service – issues e-tickets with QR codes and manages the ticket wallet. It is used by Concerts and Sports to issue tickets, by Resale to invalidate and reissue tickets, and by Venue to validate scans, and it will also serve timed-entry tickets for theatres and museums (Section 2).

> 出票服务：生成带 QR 码的电子票、管理票夹。Concerts 和 Sports 用它出票，Resale 用它作废并重发票，Venue 用它验票；以后剧场和博物馆的分时段票也用它（第 2 节）。

(c)(i) I would expose the **Event Management API** to organisers' own ticketing systems (producers): today events and price categories are entered by hand in the portal; with the API their systems can create them directly, which brings more seeds onto the platform. I would also expose the **Event Catalogue API** to media and tourism partner websites (consumers): they can list TixNow events and link fans to the purchase page, which brings more fans.

> (c)(i) 把活动管理 API 开放给主办方自己的票务系统（生产者），不用在门户手工录入，带来更多种子；把活动目录 API 开放给媒体和旅游合作网站（消费者），展示活动并引流购票，带来更多粉丝。不选 Payment：PCI-DSS，开放给外部反而增加风险。

Q1(c)(ii) Level 3 API（4 分）：以 Ticket API 为例

常见问题：只有 1 个 GET；`/wallets/` 没有 ID；status、qr-code 拆成了资源（它们是属性）；写成 `ref`（应为 `rel`）；links 里写占位符 `{ticket-no}`（应为真实值）；没有返回码、输入输出、用途；**links 不随状态变化。**若选 Ticket API，(c)(i) 要写它开放给谁（如场馆自己的闸机系统）。

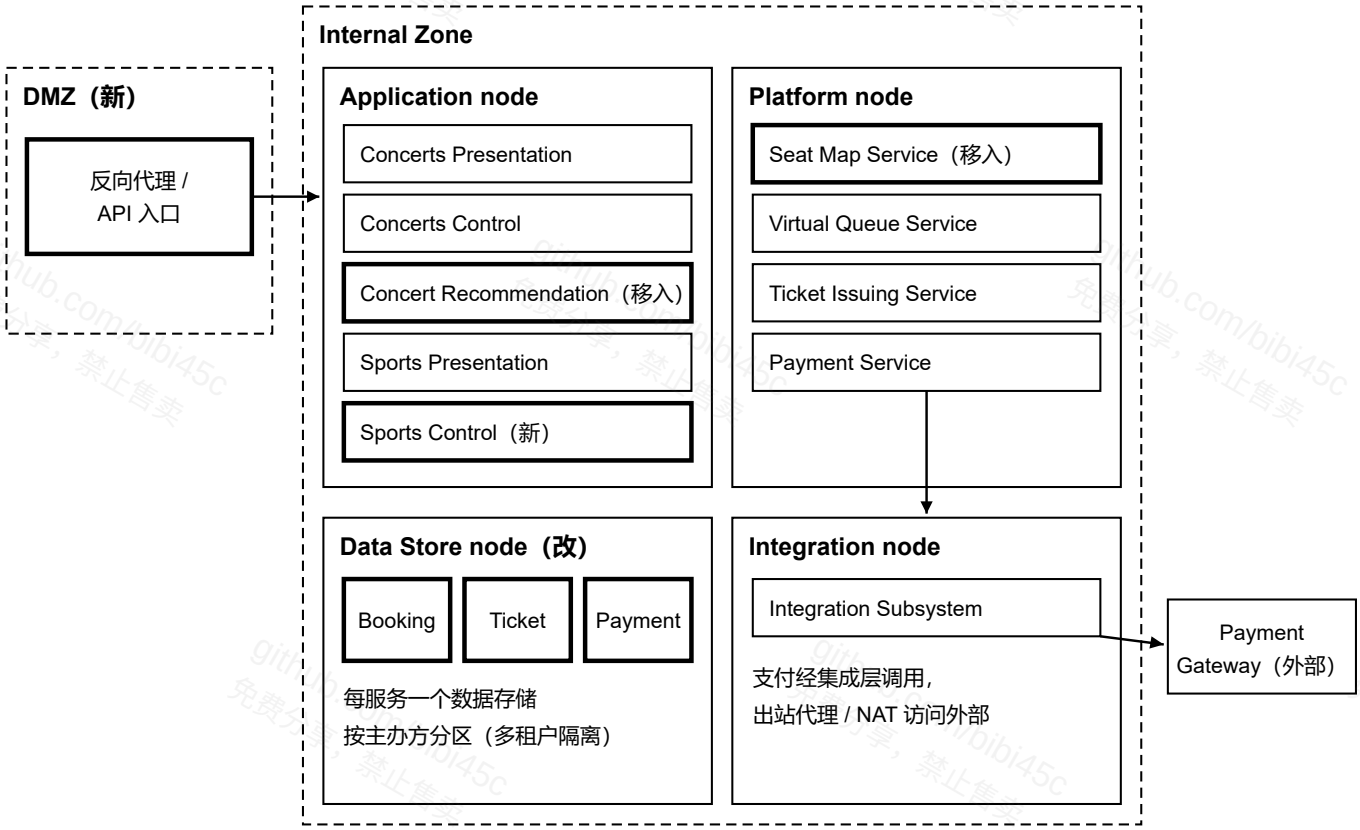
URI	HTTP	输入	输出（含 links）	用途
`/fans/{fanId}/tickets?status=valid`	GET	查询参数	200；票列表；links: self、每张票	查票夹里的有效票
`/tickets/{ticketNo}`	GET	—	200；ticketNo、eventId、seat、status、qrCode；links 随状态变	查看一张票
`/tickets/{ticketNo}/scans`	POST	gateId、scannedAt	201 Created；已用过 → 409 Conflict	闸机验票并记录入场
`/tickets/{ticketNo}`	PATCH	{"status": "invalidated"}	200；links: self	转售成交后作废旧票

```
GET /tickets/T-1001 → 200 OK（未使用）
{ "ticketNo": "T-1001", "eventId": "E-77", "seat": "A-12", "status": "valid",
  "links": [ {"rel": "self",   "href": "/tickets/T-1001"},
             {"rel": "scan",   "href": "/tickets/T-1001/scans"},
             {"rel": "resell", "href": "/resale-listings?ticketNo=T-1001"} ] }
扫码后 status = "used"：links 只剩 self 和 event（/events/E-77）
```

三、模拟卷 2 Q2(a) 图例：初稿 → 纠正后

初稿：只有 Internal Zone 1；Application node 上有 Concerts Presentation、Concerts Control、Sports Presentation、Seat Map Service；Platform node 上有 Concert Recommendation、Virtual Queue、Ticket Issuing、Payment Service（直连外部网关）；一个共用的 Data Subsystem；Integration node。

github.com/bibi45c · 免费分享，禁止售卖



粗框 = 改动处：（新）新增　（移入）从另一节点移来　（改）改了设计。Payment Service 不再直连外部网关。

错	纠正	论证
Sports **没有 Control** 子系统	加 Sports Control，部署到 Application 节点	分层要求 Presentation → Control；否则 Sports 直接调平台服务或借用 Concerts 控制层
共享的 Seat Map 在 Application 节点；只给 Concerts 用的推荐服务在 Platform 节点	两者对调	Platform 只放多个应用复用的服务；剧场扩展要复用座位图（图注原文）
Payment Service 直连外部网关，且**没有 DMZ**	加 DMZ 放反向代理；外部调用经 Integration 子系统（内部区，出站代理）	§4 DMZ 反向代理；PCI-DSS；集成层隔离外部变化
一个共用的 Data Subsystem	每服务一个数据存储，按主办方分区	§3 多租户隔离、按主办方分区

题目只要三个；最稳的三个是前三行，第四个可作备选。

github.com/bibi45c · 免费分享，禁止售卖